aCS673 SOFTWARE ENGINEERING · TEAM 3

SafeDrop

Software Test Document · Iterations 1–3

Team

Team Member	Role(s)	Signature	Date
Alexa Stein	Team Lead	*Alexa Stein*	*10/12/2026*
Amber Rastella	Security Lead	*Amber Rastella*	*10/12/2026*
Alex Picard	Configuration Lead	*Alex Picard*	*10/12/2026*
Mateus Silva	Requirements Lead	*Mateus Silva*	*10/12/2026*
Orelmis Toribio	QA Lead	*Orelmis Toribio*	*10/12/2026*

Revision History

Version	Author	Date	Change
0.1	Amber (draft)	9/21/2026	Initial Iteration 1 test document. Automated testing report and metrics compiled from the CI run on main at commit e4a0efd. The manual testing report is not yet filled in - see the note in that section.
0.2	Orelmis Toribio (QA Lead)	9/21/2026	Added code-structure metrics (average LOC per file, LOC per function, internal fan-out per file) to Testing Metrics. Added manual tests, and styling.
0.3	Orelmis Toribio (QA Lead)	10/4/2026	Added Iteration 2: testing summary, manual test cases MT-19 to MT-37 (results to be recorded by the QA Lead when run), automated testing changes, and test and code-structure metrics from the CI run on main at commit 5c4bcc8. Sections 1, 2 and 4 now have separate Iteration 1 and Iteration 2 subsections, with an Iteration 1 vs Iteration 2 comparison in 4.3.

			Iteration 1 content is unchanged.
0.4	Orelmis Toribio (QA Lead)	10/11/2026	Added Iteration 3: testing summary, manual test cases MT-36 to MT-48 (results to be recorded by the QA Lead when run), automated testing changes, and test and code-structure metrics at commit ebfae65 (main, 11 October 2026), with an Iteration 2 vs Iteration 3 comparison in 4.5. Figures are from runs of that commit's backend and frontend suites with coverage; its CI run passed every job. Iteration 1 and 2 content is unchanged.

Contents

1. Testing Summary
2. Manual Testing Report
3. Automated Testing Report
4. Testing Metrics
5. AI Usage Log
6. References
7. Glossary

1. Testing Summary

1.1 Iteration 1

Iteration 1 was tested almost entirely through automated tests, run on every push and pull request by GitHub Actions and required to pass before a branch may merge to main. The QA Lead owns the suite as a whole; each developer writes the tests for the feature they build, and the reviewer checks test coverage as part of the code review described in the SPPP. The figures throughout this document come from the CI run on main at commit e4a0efd (21 September 2026), which passed all six jobs.

921 test cases passed	**100%** pass rate (executed cases)	**94.6% / 96.0%** line coverage, backend / frontend	**0** high or critical vulnerabilities

What was tested, by category:

Category	What was tested

Unit testing	Services, middleware, policies and utilities, with their dependencies mocked. 20 of the 32 backend test files are unit tests; the largest is the checkout state machine (72 cases covering every legal and illegal transition). The frontend unit tests render components with React Testing Library against a mocked API layer.
Integration testing	The Express API exercised end to end through Supertest against a real MongoDB replica set started in memory for the run, so transactions, unique constraints and TTL indexes behave as they do in production rather than being mocked. 12 backend test files are integration tests.
System testing	The full stack (SPA, API and a MongoDB replica set) is brought up with Docker Compose and exercised by hand against seeded data. CI additionally builds both Docker images on every push, so a change that breaks either image fails before it merges.
Acceptance testing	Acceptance criteria for work not yet implemented are staged in the suite as bodyless test cases (see code/docs/staged-acceptance-tests.md), so a story's definition of done lives somewhere that is run and reviewed rather than in prose that can drift. They are replaced by real tests when the ticket is implemented.
Regression testing	The whole suite runs on every push and pull request, so every previously passing test is a regression test. Two security regressions found during Iteration 1 were each closed with a test that fails without the fix (SCRUM-114 and the tenant-isolation cases below).

1.2 Iteration 2

Iteration 2 (from the Iteration1 release tag on 21 September to commit 5c4bcc8 on 4 October 2026) was tested the same way: the whole suite runs on every push and pull request, and a branch merges only with CI green. The figures below come from the CI run on main at commit 5c4bcc8 (4 October 2026), which passed every job.

1,508 test cases passed	**100%** pass rate (executed cases)	**95.2% / 97.8%** line coverage, backend / frontend	**0** high or critical vulnerabilities

What Iteration 2 added, and where it is tested:

Story	What was built, and how it is tested
Unit maintenance (SCRUM-141)	An Org Admin moves an AVAILABLE unit into

	MAINTENANCE and back; any other unit state is refused with 409, and the dashboard counts units in maintenance. New: assetMaintenance.test.js (18 cases), including cross-tenant 404s and race safety.
Overdue loans (SCRUM-140, SCRUM-143)	Late CHECKED_OUT loans are moved to OVERDUE by an admin-only endpoint, and borrowers see "Due in N days" / "Overdue by N days" badges. New: overdue.test.js (23), markOverdue.test.js (11), and the frontend DueBadge and dueStatus tests.
Auto-approve (SCRUM-148)	An Org Admin decides whether requests need an approver, organization-wide with a per-asset override; an auto-approved request goes straight to APPROVED with its unit HELD. New: approvalSettings.test.js (30), plus new cases in approvalPolicy.test.js.
User groups and restricted equipment (SCRUM-149, SCRUM-150)	Org Admins create groups and restrict assets to them; an ineligible member sees a Restricted badge and is refused on submit and on approval. New: groups.test.js (41), group.test.js (13), allowedGroupIds.test.js (5), plus new cases in the assets and requests tests.
Custody confirmation (SCRUM-205)	The borrower can record their own pickup; a return waits in RETURN_PENDING until someone other than the borrower confirms or rejects it; approvals nobody collects expire after a configurable pickup window. New: custodyConfirmation.test.js (51).
Catalogue search (SCRUM-103, -145, -146, -200, -201)	Plain and AI-assisted (Microsoft Foundry) search, retired assets excluded and results capped; any model failure falls back to plain search. New: assetSearch unit (19) and route (11) tests, plus CatalogPage tests. The model is mocked in every automated case.
AI-recommended alternatives (SCRUM-151)	"Similar items available now" on an asset with nothing available, applying the same eligibility filters as the AI path, with a non-AI fallback. 29 cases (22 backend, 7 frontend), itemised in code/docs/std-scrum-151-test-cases.md.
Other changes	A deactivated member is refused at sign-in with the same 401 as a wrong password (deactivate.test.js). Iteration 1 review feedback split the Members page into InviteForm and ResetPasswordForm, and the UI rework restyled every page. PR #64 fixed approval-queue rows showing "Unknown user" and "Unknown asset", with 9 regression cases in requests.test.js.

1.3 Iteration 3

Iteration 3 (from the Iteration2 release tag on 5 October to commit ebfae65 on 11 October 2026: 37 commits, PRs #69 to #82) was tested the same way: the whole suite runs on every push and pull request, and a branch merges only with CI green. The CI run on main at ebfae65 passed every job: lint, format check, tests with coverage and npm audit for both packages, both Docker builds, and the staging deploy. The figures below come from runs of that commit's backend and frontend suites with coverage (see 4.4.1).

1,645 test cases passed	**100%** pass rate (executed cases)	**95.7% / 96.3%** line coverage, backend / frontend	**0** high or critical vulnerabilities

What Iteration 3 added, and where it is tested:

Story	What was built, and how it is tested
Search telemetry (SCRUM-206)	Every AI search and every similar-items lookup writes one SearchQueryLog row: outcome, fallback reason, latency and counts, with no query text and no user id (OD-9). The write is fire-and-forget, so it cannot fail or slow a search, and rows expire after 90 days through a TTL index. Org Admins read aggregates for their own organization only, from GET /api/search-telemetry. New: searchTelemetry.test.js (integration) and searchTelemetry.test.js and searchTelemetry.recording.test.js (unit), covering no reconstructable query text, admin-only access, a smuggled orgId ignored (SR-2), the retention index, and searches that survive a failed write.
AI search page and labels (SCRUM-242)	A new admin page, AI search (under Manage), shows a status line, a 30-day chart of searches stacked by outcome, headline figures and why searches fell back. The Overview shows an alert only when AI search needs attention: failures at 20% or more of at least 5 searches, or a search cut off at the 100-candidate limit. Catalog results say "Ranked by AI" or "Showing text matches", and similar items say "Suggested by AI" when the model chose them. New: AiSearchPage (10), AiSearchAlert (3) and aiSearchStatus (9) frontend tests, plus new cases in CatalogPage, SimilarItems and the telemetry route tests.
AI search quality and resilience (SCRUM-243, SCRUM-244)	Sentence queries now reach the right items: the candidates sent to the model are phrase matches, then assets scored by how many of the query's words they contain. A fast Foundry failure (network error, 429 or 5xx) is retried once within the same deadline, honouring Retry-After, and the plain-search fallback also matches individual

	words instead of showing "No assets match". The Foundry timeout rose from 10 s to 15 s and the catalog search debounce from 300 ms to 600 ms. A demo catalogue seed (npm run seed:catalog, 90 assets by default) exercises AI search at realistic size. New cases in assetSearch.test.js and foundryClient.test.js; the model stays mocked in every automated case.
Relevant alternatives (SCRUM-184)	Similar items now draw candidates from the stuck asset's own category first and ask the model for a substitute; with AI off or failing, the fallback offers the same category only (AT-3 as written, reversing the Iteration 2 deviation). Six new cases in assetAlternatives.test.js (21 in the file), itemised in the SCRUM-184 addendum to code/docs/std-scrum-151-test-cases.md.
Request dates in local time (SCRUM-240)	The request form now sends the chosen days as the start of the first and the end of the last day in the member's own time zone, so a request no longer displays a day early and a loan falls due at the end of its last day. This fixes the MT-21 partial result from Iteration 2. Tested in the validateWindow, format, RequestCreation and RequestDetailPage tests (including both daylight-saving changes) and a backend check that the instants are stored unchanged.
UI redesign (SCRUM-241)	Requests as a board (Waiting for a decision, Ready for pickup, Out on loan, Done) with one-click actions on each card, a loan timeline on the Overview, a ⌘K / Ctrl+K command palette, gallery cards in the catalog, and side panels (drawers) for inviting a member, setting a password and creating a group. New: LoanTimeline (5); the rest is tested through the rewritten page tests (MembersPage, GroupsPage, ApprovalQueuePage, Layout and others). Drawer, RequestBoard and CommandPalette have no test file of their own and are the least covered frontend files (4.4.1).
Other changes	SCRUM-225 removed duplicated code from checkout.service.js without changing behaviour; the checkout state-machine tests passed unchanged. SCRUM-216 completed the AI-usage headers on source and test files. SCRUM-191 was a research spike with no code change (OD-10, code/docs/od-10-ai-candidate-retrieval.md). Dependabot bumped shell-quote and concurrently (PR #71). A manual "Seed demo catalog (staging)" workflow runs the catalogue seed against staging, dry-run by default.

2. Manual Testing Report

2.1 Approach

Manual testing during Iterations 1 and 2 was exploratory: features were exercised by hand against a locally seeded stack as they were built, and results were not recorded as structured test cases at the time. The QA lead reperformed the following manual tests after the fact as a final check

2.2 Test environment

The full stack runs locally with Docker Compose. From the repository root, run docker compose up --build; copying .env.example to .env is optional, since every variable has a development default. This starts four services:

Service	Role in manual testing
mongo	A single-node MongoDB replica set. A replica set is required because the backend wraps every state-changing action in a transaction, so each state change and its audit event commit together or not at all.
mongo-init	A one-shot sidecar that initiates the replica set once Mongo is healthy, then exits.
backend	Runs migrations, then the API at http://localhost:4000. /health returns 200 when the API and database are up and 503 when the database is unreachable.
frontend	The Vite dev server at http://localhost:5173, with /api/* proxied to the backend so the browser talks to a single origin.

Email. Locally nothing is emailed: MAIL_PROVIDER defaults to console, so password-reset emails (subject, body and reset link) are printed to the backend log. Password-reset tests read the link from docker compose logs -f backend.
Seeding. With the stack running, run npm run seed from code/backend. This calls seedTwoOrgs() (tests/helpers/seedTwoOrgs.js), the same fixture the integration suite uses, plus demo-only chain-of-custody history. The seed is destructive: it drops and recreates the two demo organizations on every run without prompting, and it refuses to run under NODE_ENV=production. Cases that change state should be followed by a re-seed before the next case.

2.3 Seed dataset

The seed creates two identically built organizations, Org A (org-a) and Org B (org-b), so every check against one has an equivalent to compare against the other. Each organization contains the following.
Users (all share the password Correct-Horse-Battery-9):

Role	Org A	Org B	Can do
MEMBER	member@a.test	member@b.test	Browse the catalogue, submit and cancel their

			own requests, see only their own requests.
APPROVER	approver@a.test	approver@b.test	Everything a member can, plus approve and deny requests and record checkout and return handoffs.
ORG_ADMIN	admin@a.test	admin@b.test	Everything an approver can, plus create, edit and retire assets, add units, manage member roles, read the audit log and read the dashboard.

Assets and units (for each organization):

Asset	Units	Unit statuses
Laptop	3	REQUESTED, OUT, HELD
Camera	3	AVAILABLE, AVAILABLE, RETIRED
Projector	2	AVAILABLE, OUT

Checkout requests (all raised by the organization's member):

Request status	Unit
PENDING	The laptop's REQUESTED unit
APPROVED	The laptop's HELD unit
CHECKED_OUT	The laptop's OUT unit (due 2026-09-24 or 2026-09-26)
CHECKED_OUT	The projector's OUT unit (due 2026-09-24 or 2026-09-26)
LOST	The camera's RETIRED unit; the request moved PENDING → APPROVED → CHECKED_OUT → LOST

Audit trail. One ORG_CREATED event from the fixture, plus nine chain-of-custody events that the seed script appends to the laptop, so its history screen shows a populated, coherent timeline.

2.4 Iteration 1 Manual test cases

ID	Test case	Type	Priority	Result
MT-01	Sign in and sign out for every role	System	High	Pass
MT-02	Create a new organization	Acceptance	High	Pass
MT-03	Forgot password,	Acceptance	Medium	Pass

	reset and change password			
MT-04	Verify role based access to actions (admin, approver, member)	System (security)	High	Pass
MT-05	Verify Tenant isolation between organizations (assets, requests etc.)	System (security)	High	Pass
MT-06	Browse and filter the asset catalogue	Acceptance	Medium	Pass
MT-07	Create and edit an asset, add a unit	Acceptance	High	Pass
MT-08	Retire an asset (with no active units)	Acceptance	High	Pass
MT-09	Retire an asset (blocked by active units)	Acceptance	High	Pass
MT-10	Submit a checkout request; duplicate refused	Acceptance	High	Pass
MT-11	Approve and deny a request	Acceptance	High	Pass
MT-12	Record checkout and return handoffs	Acceptance	High	Pass
MT-13	Cancel a pending and an approved request	Acceptance	Medium	Pass
MT-14	View My Requests (member view)	Acceptance	Medium	Pass
MT-15	View Admin dashboard summary	Acceptance	Medium	Pass
MT-16	View Asset History	Acceptance	Medium	Pass
MT-17	Audit log filter, search and pagination	Acceptance	Medium	Pass

MT-18	Change a member's role	Acceptance	Medium	Pass

2.5 Iteration 2 Manual test cases

The cases below cover the features merged in Iteration 2. They use the same environment and seed as Sections 2.2 and 2.3 (the seed script did not change in Iteration 2); any extra setup is stated in the case. Re-seed before each case that changes state.
Dates. The seed's dates are fixed, so from 4 October 2026 both CHECKED_OUT loans are past due and the APPROVED laptop is past its default 48-hour pickup window. MT-21, MT-22 and MT-31 rely on this.

2.5.1 Test case summary

ID	Test case	Type	Priority	Result
MT-19	Put a unit into and out of maintenance	Acceptance	High	Partial - There seems to be a delay with the “Available” counter on the dashboard after maintenance modification
MT-20	Maintenance guarded: wrong unit state, stale page, non-admin roles	System (security)	Medium	Pass
MT-21	Due-date badges on borrowed items	Acceptance	Medium	Partial - Difference in timezone between request date input, what's stored, whats displayed
MT-22	Mark late checkouts overdue	Acceptance	High	Pass
MT-23	Approval queue: Overdue filter and readable rows (PR #64)	Regression	High	Pass
MT-24	Organization-wide auto-approve	Acceptance	High	Pass
MT-25	Per-asset approval override	Acceptance	High	Pass
MT-26	Create, rename and delete a user group; add and remove members	Acceptance	High	Pass

MT-27	Restricted equipment: members outside and inside the group	System (security)	High	Pass
MT-28	Borrower records their own pickup	Acceptance	High	Pass
MT-29	Return confirmed or rejected by someone other than the borrower	Acceptance	High	Pass
MT-30	An approver cannot confirm their own return	System (security)	High	Pass
MT-31	Pickup window and expiring uncollected approvals	Acceptance	Medium	Pass
MT-32	Catalogue search (plain search, retired assets excluded)	Acceptance	Medium	Pass
MT-33	Similar items available now	Acceptance	Medium	Pass
MT-34	Tenant isolation on the Iteration 2 pages (groups, alternatives, search)	System (security)	High	Pass
MT-35	Members page: invite a member and reset a password	Regression	Medium	Pass

2.5.2 Test steps

ID	Steps	Expected result
MT-19	1. Sign in as admin@a.test and open Dashboard; note the In maintenance and Available figures. 2. Open Catalog, then Camera. 3. In the Units table, click Start maintenance on one AVAILABLE unit. 4. Go back to Dashboard.	Step 3: a notice says the unit is now in maintenance and cannot be requested; its status shows MAINTENANCE and it no longer offers Request this. Step 4: In maintenance is up by 1 and Available down by 1. Step 5: the unit is AVAILABLE again and can be requested.

	5. Return to Camera and click End maintenance on the same unit. 6. Open Audit log and find the newest events for the camera.	Step 6: one event for entering maintenance and one for leaving it, each naming the admin.
MT-20	1. As admin@a.test, open Laptop and look at the Maintenance column. 2. Open Camera in two browser tabs. In tab 1 click Start maintenance on an AVAILABLE unit; then, without refreshing, click Start maintenance on the same unit in tab 2. 3. End maintenance on that unit, then sign in as approver@a.test and as member@a.test and open Camera.	Step 1: units that are REQUESTED, OUT or HELD show a dash, with no maintenance button. Step 2: tab 2 shows an error that the unit is not available (409); after a refresh the unit is in maintenance once, with a single audit event. Step 3: there is no Maintenance column for an approver or a member.
MT-21	1. Sign in as member@a.test and open My requests. 2. Open the CHECKED_OUT laptop request and the CHECKED_OUT projector request. 3. Request an AVAILABLE camera unit from today to tomorrow; approve it as approver@a.test, then record the handoff. 4. As member@a.test, open My requests again.	Steps 1-2: both seeded loans (due 24 and 26 September) show "Overdue by N days", with N matching today's date; requests that are not out (pending, approved, lost) show no due badge. Step 4: the new loan shows "Due tomorrow".
MT-22	1. Sign in as admin@a.test. Open Dashboard and note the Overdue figure, then open Approvals and choose the Overdue filter. 2. In the browser console run: await fetch('/api/requests/mark-overdue', { method: 'POST', credentials: 'include' }).then(r => r.json()) 3. Run the same command again. 4. Refresh Approvals and open both overdue requests. 5. Sign in as approver@a.test and run the command from step 2.	Step 1: Overdue is 2 and both loans (laptop and projector) are listed under Overdue, even though their stored state is still Checked out. Step 2: the response is {"updated": 2}. Step 3: the response is {"updated": 0} - running it twice changes nothing. Step 4: both requests now show the state OVERDUE; the LOST camera request is unchanged. Step 5: the request is refused with 403.
MT-23	1. Sign in as approver@a.test and open Approvals.	Step 2: every row names its requester, asset and unit - no row

	2. Click through each state filter: All, Pending, Approved, Checked out, Overdue, Return pending, Returned, Denied, Cancelled, Expired, Lost.	shows "Unknown user" or "Unknown asset" - and each filter lists only requests in that state (Overdue lists late loans whatever their stored state).
MT-24	1. Sign in as admin@a.test, open Settings, choose Approve automatically under Checkout approval and save. 2. Sign in as member@a.test and request an AVAILABLE camera unit. 3. Open the new request, then Camera's unit list. 4. As admin@a.test, open Audit log. 5. Set Settings back to Require an approver and submit another request as member@a.test.	Step 3: the request is APPROVED straight away with the note "Approved automatically by your organization's rules", and the unit is HELD. It still needs a pickup to be recorded. Step 4: REQUEST_SUBMITTED followed by REQUEST_AUTO_APPROVED naming the policy, plus an ORG_SETTINGS_UPDATED event for the settings change. Step 5: the new request is PENDING and appears in the approval queue.
MT-25	1. Leave the organization default as Require an approver. As admin@a.test, edit Projector and set Checkout approval to Always approve automatically; save. 2. As member@a.test, request the AVAILABLE projector unit. 3. Set the organization default to Approve automatically and set Camera to Always require an approver; request a camera unit as member@a.test. 4. Set Camera back to Use the organization default.	Step 2: the projector request is APPROVED immediately; the asset page shows its approval setting. Step 3: the camera request is PENDING despite the organization default - the per-asset setting wins. Step 4: the asset follows the organization default again.
MT-26	1. Sign in as admin@a.test, open Groups and create a group named "Field Team". 2. Try to create another group named "field team". 3. Open Field Team, add member@a.test with Add to group, then rename the group to "Field Crew". 4. Remove member@a.test, then delete the group. 5. Open Audit log.	Step 1: the group appears in the list with no members. Step 2: refused - group names are unique within an organization regardless of letter case. Step 3: the member is listed and the new name shows everywhere the group appears. Step 4: the member is removed and the group disappears from the list. Step 5: one GROUP_ event for each change, naming the admin.
MT-27	1. As admin@a.test, create a group "Camera Club" with approver@a.test as its only	Step 3: Camera is still listed but shows a Restricted badge; its page says "Only members of

	member. 2. Edit Camera and, under Who can request this, restrict it to Camera Club. 3. Sign in as member@a.test and open the Catalog, then Camera. 4. Sign in as approver@a.test and request an AVAILABLE camera unit. 5. As admin@a.test, add member@a.test to Camera Club; as member@a.test, reload Camera.	Camera Club can request this" and every Request button is disabled. Step 4: the request is accepted. Step 5: the member can now request a camera unit.
MT-28	1. As member@a.test, open the seeded APPROVED laptop request (or approve a new camera request as approver@a.test). 2. Click "I've picked it up". 3. As approver@a.test, open the same request in a tab that was opened before step 2 and click Record handoff. 4. Invite a second member (see MT-37) and, signed in as them, open the first member's request URL.	Step 2: the request becomes CHECKED_OUT, the unit OUT, and a due date is set; the history shows the pickup was recorded by the borrower. Step 3: refused with a message that the item is already checked out - the pickup is recorded once. Step 4: the second member cannot see the request and has no pickup action.
MT-29	1. As member@a.test, open the CHECKED_OUT projector request and click Return this item, adding a condition note. 2. As approver@a.test, open Approvals with the Return pending filter, open the request and click Confirm return. 3. Re-seed. Repeat step 1, then as approver@a.test click Reject return instead. 4. As approver@a.test, use Record return on the CHECKED_OUT laptop (a walk-in return).	Step 1: the request is RETURN_PENDING and the unit stays OUT; the member has no Confirm button. Step 2: the request is RETURNED and the unit AVAILABLE; the page shows the condition received. Step 3: the request goes back to CHECKED_OUT and the unit stays OUT. Step 4: the request is RETURNED in one step.
MT-30	1. As approver@a.test, request an AVAILABLE camera unit. 2. As admin@a.test, approve it. 3. As approver@a.test, click "I've picked it up", then Return this item. 4. Still as approver@a.test, open the request.	Step 4: the request is RETURN_PENDING and the approver is offered neither Confirm return nor Reject return on their own loan. Step 5: the admin can confirm the return.

	5. As admin@a.test, open the request.	
MT-31	1. Sign in as admin@a.test and open Settings. Note the Pickup window (Hours to collect an approved item). 2. Click Expire uncollected approvals now. 3. Click it again. 4. Open Laptop, then Approvals with the Expired filter. 5. Enter 721 in the hours field, then 24, and save.	Step 1: the window is 48 hours by default. Step 2: "1 approval expired and the units released." - the seeded laptop approval (needed from 25 September) is past its window. Step 3: nothing more expires. Step 4: the laptop's HELD unit is AVAILABLE and the request is EXPIRED, with a system audit entry. Step 5: 721 cannot be saved (whole hours, up to 720); 24 saves with a confirmation.
MT-32	1. As member@a.test, open Catalog and type "cam" into Search the catalog without pressing Enter. 2. Change the text to a word that matches nothing. 3. As admin@a.test, create an asset "Tripod" with no units and retire it. 4. As member@a.test, search for "tripod".	Step 1: "Searching..." appears briefly, then Camera is listed; results follow the typing without a submit. Step 2: an empty result, not an error. Step 4: no results - retired assets are left out of search.
MT-33	1. As member@a.test, open Laptop (it has no AVAILABLE unit). 2. Open Camera (it has AVAILABLE units). 3. As admin@a.test, restrict Projector to a group member@a.test is not in; reload Laptop as member@a.test.	Step 1: a "Similar items available now" section lists up to five available assets (not the laptop itself), each with a reason (Foundry on) or "Available now" (Foundry off) and a View link. Step 2: no section is shown. Step 3: Projector is no longer suggested.
MT-34	1. As admin@a.test, create a group and copy its URL; copy the Laptop asset URL too. 2. Sign in as admin@b.test and open both URLs. 3. As admin@b.test, open Groups and search the catalog for "laptop".	Step 2: both show not found - no Org A data and no Org A similar items. Step 3: only Org B's groups and assets are listed.
MT-35	1. As admin@a.test, open Users and invite a new member with a name, email and initial password. 2. Sign in as the new member. 3. As admin@a.test, click Reset	Step 1: the member appears in the list with the MEMBER role. Step 2: they are asked to set a new password before using the app.

	password for that member and note the temporary password. 4. Sign in as the member with the temporary password.	Step 4: the temporary password works once, and they must set a new password again.

2.6 Iteration 3 Manual test cases

2.6.1 Test case summary

ID	Test case	Type	Priority	Result
MT-36	Request dates stay on the chosen days (time-zone fix)	Regression	High	Pass
MT-37	Telemetry records how a search behaved, never its text or who searched	System (security)	High	Pass
MT-38	AI search page: status, chart, figures, and who can open it	Acceptance	Medium	Pass
MT-39	Catalog search labels and the word-matching fallback	Acceptance	High	Pass
MT-40	Demo catalogue seed: dry run, seed, re-run and remove	Acceptance	Low	Parital: The commands do what they're supposed to. (Nothing wrong with the new code). However, this reveals assets page does not have proper pagination implemented. Only the first 25 seeded assets can be seen.
MT-41	AI search over the demo catalogue (Foundry on)	Acceptance	Medium	Pass
MT-42	Similar items stay in the stuck asset's	Acceptance	High	Pass

	category (Foundry off)			
MT-43	Request board: columns and one-click actions by role	Acceptance	High	Pass
MT-44	Loan timeline on the Overview	Acceptance	Medium	Pass
MT-45	Command palette offers only the viewer's pages	System (security)	Medium	Pass
MT-46	Side panels: invite a member, set a password, create a group	Acceptance	Medium	Pass
MT-47	Tenant isolation on AI search figures	System (security)	High	Pass
MT-48	Re-run MT-19: dashboard counts after maintenance	Regression	Medium	Partial (No iteration 3 story resolved the bug, delay with dashboard updates)

2.6.2 Test steps

ID	Steps	Expected result
MT-36	1. With the browser set to America/New_York, sign in as member@a.test and request an AVAILABLE camera unit from today to tomorrow. 2. Before submitting, open the browser's network panel; submit and inspect the POST body. 3. Open the new request, then My requests. 4. As approver@a.test, open Approvals; approve the request and record the handoff. 5. As member@a.test, open My requests again.	Step 2: neededFrom is the local start of today and neededTo the local end of tomorrow, as UTC instants (in October, T04:00:00.000Z and T03:59:59.999Z the following day). Step 3: both pages show today to tomorrow, not the day before. Step 4: Approvals shows the same two days. Step 5: the loan shows "Due tomorrow".
MT-37	1. Re-seed. As member@a.test, search the catalog for "cam", then open Laptop (no AVAILABLE unit, so a similar-items lookup runs). 2. Run: docker compose exec	Step 2: one row per search and one for the Laptop lookup (kind alternatives, with the laptop's assetId). Each row has orgId, timestamp, kind, aiAssisted, fallbackReason ("disabled" with

	mongo mongosh safedrop --quiet --eval 'db.searchquerylogs.find().sort({timestamp:-1}).limit(5).toArray()' 3. As member@a.test, in the browser console run: await fetch('/api/search-telemetry', { credentials: 'include' }).then(r => r.status) 4. Repeat step 3 as approver@a.test.	Foundry off), latencyMs, candidateCount and resultCount, and nothing else: no "cam", no hash of it and no user id. Steps 3-4: 403 for both.
MT-38	1. Re-seed. As admin@a.test, open AI search under Manage. 2. As member@a.test, search the catalog three times. 3. As admin@a.test, reload AI search. 4. Open the chart's table view and hover a day. 5. As member@a.test and approver@a.test, look for AI search in the navigation, then open /admin/ai-search directly.	Step 1: the status reads "No searches yet". Step 3 (Foundry off): "AI search is switched off", stated as a fact, not a warning; Searches shows 3; "Why searches fell back" accounts for all three. Step 4: the table lists the same per-day counts as the bars. Step 5: the page is not offered and cannot be opened.
MT-39	1. With Foundry off, as member@a.test type "cam" into Search the catalog. 2. Replace it with "a camera for the weekend". 3. Turn Foundry on (valid key) and search "something to film a talk".	Step 1: Camera is listed under "Showing text matches for "cam"", with no "Ranked by AI". Step 2: Camera is still listed (word match), not "No assets match". Step 3: placeholder cards appear; after about 1.5 s the message says the AI is comparing the request with the catalog; results then show "Ranked by AI"
MT-40	1. From code/backend run npm run seed:catalog -- --dry-run. 2. Run npm run seed:catalog. 3. Run it again. 4. As member@b.test, open the Catalog. 5. Run npm run seed:catalog -- --remove.	Step 1: reports org-b's asset count, the 90 assets it would add and whether the total crosses the 100-item AI limit; nothing is written. Step 2: 90 assets are added across 15 categories. Step 3: nothing is added the second time. Step 4: the demo assets are listed alongside the seeded three. Step 5: exactly the demo assets and their units are removed; Laptop, Camera and Projector remain.

MT-41	1. Seed the demo catalogue (MT-41 step 2) and turn Foundry on. 2. As member@b.test, search "something to record an interview". 3. Search "something to draw digital art on". 4. As admin@b.test, open AI search.	Step 2: the Zoom recorders are at or near the top, "Ranked by AI", each with a reason, within 15 s. Step 3: the Wacom tablets are at or near the top. Step 4: both searches counted as ranked by AI; no catalog-size warning (90 assets is under the limit).
MT-42	1. Re-seed. With Foundry off, as member@a.test open Laptop. 2. As admin@a.test, create an asset "Spare Laptop" in category laptop with one unit. 3. As member@a.test, reload Laptop.	Step 1: no "Similar items available now" section, although the camera and projector have AVAILABLE units: neither is a laptop. Step 3: the section lists Spare Laptop with "Available now" and nothing else; no "Suggested by AI" marker.
MT-43	1. Re-seed. As approver@a.test, open Approvals. 2. Click Approve on the pending laptop card. 3. As member@a.test, open My requests. 4. Click "I've picked it up" on the approved laptop card.	Step 1: four columns: Waiting for a decision, Ready for pickup, Out on loan, Done. The pending card offers Approve and Deny; the approved card offers Record handoff; loan cards offer no return button, only a link to the request. Step 2: the card moves to Ready for pickup. Step 3: the member's board shows only their requests; a pending card offers Cancel request, never Approve. Step 4: the card moves to Out on loan with a due badge.
MT-44	1. Re-seed. As approver@a.test, deny the pending laptop request. 2. As admin@a.test, open the Overview. 3. Click a row.	Step 2: the loan timeline shows the live requests against a line for today; both overdue loans run past their due date to today in the danger colour; the denied request is not shown. Step 3: the request's page opens.
MT-45	1. As member@a.test, press Ctrl+K (⌘K on a Mac). 2. Type "cam", move with the arrow keys and press Enter. 3. Press Ctrl+K, then Escape. 4. As admin@a.test, press Ctrl+K and type "ai".	Step 1: the palette lists only pages a member can open: no Users, Groups, Settings, Audit log or AI search. Step 2: Camera opens. Step 3: the palette closes and focus returns to where it was.

		Step 4: AI search is offered.
MT-46	1. As admin@a.test, open Users and click the invite button. 2. Press Tab repeatedly, then Escape. 3. Open it again, type a name, close it with the Close button, and open it once more. 4. Click Reset password for a member; set a password. 5. Open Groups and create a group from its panel.	Step 1: an "Invite a member" panel slides in from the right with focus in its first field. Step 2: focus stays inside the panel; Escape closes it and focus returns to the invite button. Step 3: the form is empty again. Step 4: a "Set a password for <name>" panel opens; saving works as in MT-35. Step 5: a "Create a group" panel opens and the new group is listed.
MT-47	1. Re-seed. As member@a.test, search the catalog three times. 2. As admin@b.test, open AI search. 3. As admin@b.test, in the console run: await fetch('/api/search-telemetry?orgId=anything', { credentials: 'include' }).then(r => r.json())	Step 2: "No searches yet": Org A's searches are not counted. Step 3: Org B's figures (total 0); the orgId parameter is ignored.
MT-48	Repeat MT-19 exactly.	As MT-19. Record whether the dashboard's Available figure now changes as soon as the page is reopened. No Iteration 3 change targeted this, so a Partial result here keeps the defect open.
MT-49	1. Open the staging frontend and backend /health using http:// instead of https://. 2. Inspect the response headers of any API call.	Step 1: both redirect to https:// or refuse; no page or data is served over http. Step 2: a Strict-Transport-Security header is present.

3. Automated Testing Report

Where the tests live

Backend tests are in code/backend/tests, split into tests/unit (services, middleware, policies, utilities) and tests/integration (routes, plus a security folder). Frontend tests are in code/frontend/tests/unit, covering components, hooks, services and utilities. Frameworks: Vitest throughout; Supertest for API routes without starting an HTTP server; React Testing Library for components; mongodb-memory-server to run a real MongoDB replica set in memory for the duration of a run. Each backend test file gets its own database inside that replica set, with the production indexes applied by the same migrations that run in deployment, and the data is wiped between tests. Playwright end-to-end tests remain a nice-to-have and were not written in Iteration 1.

How it runs

Every push and pull request runs, for both backend and frontend: lint, a formatting check, the test suite with coverage, and npm audit; plus a Docker image build for each. Coverage is gated - a run fails if line coverage falls below 80%, the target set in the SPPP quality assurance plan (NFR-10/11). Three test files are worth calling out because they are what makes the security design checkable rather than merely stated:

Test file	Cases	What it proves
tests/integration/security/routePermissions.test.js	134	Walks the built route table and checks every registered route against the permission matrix, including that each returns 401 without a session. A newly added route cannot be left unprotected by accident (SR-1).
tests/integration/security/tenantIsolation.test.js	27	Seeds two organizations and confirms a token from one gets 404 - not 403 - for the other's ids, at both the repository and the route layer (SR-2). All 11 endpoint rows are active; six that were still flagged unimplemented after their endpoints had shipped were enabled during Iteration 1, and all six passed.
tests/unit/services/checkout.transitions.test.js	72	Every legal and illegal transition of the checkout state machine, including what each does to the physical unit.

Iteration 2 changes and key test files

The layout and frameworks are unchanged. The backend suite grew from 32 to 45 test files (19 unit, 26 integration) and the frontend from 28 to 36. Tests never inherit a live Foundry key: the Vitest configuration forces FOUNDRY_ENABLED=false and an empty key, so a test that forgot to mock the client cannot make a real, billed call, and the AI paths are tested against a recorded response (tests/fixtures/foundry/asset-search-response.json). The backend Docker image gained a dev stage so the backend suite can also run inside the container.
The three files above at Iteration 2, and the largest new test files:

Test file	Cases	What it proves
tests/integration/security/routePermissions.test.js	229	134 cases in Iteration 1. The new group, settings, maintenance, search, alternatives and custody routes were picked up by the route-table walk with no change to the test, so none of them can ship unprotected (SR-1).
tests/integration/security/tenantIsolation.t	27	Unchanged in size. Cross-tenant

est.js		404s for the new endpoints are checked in each feature's own tests (maintenance, groups, alternatives) (SR-2).
tests/unit/services/checkout.transitions.test.js	108	72 cases in Iteration 1. The table-driven cases grew with the state machine, which gained OVERDUE, RETURN_PENDING and EXPIRED.
tests/integration/routes/custodyConfirmation.test.js	51	Pickup recorded by either side and only once, returns confirmed or rejected by someone other than the borrower (with the sole-confirmer fallback), and expiry of uncollected approvals (SCRUM-205).
tests/integration/routes/groups.test.js	41	Group lifecycle, case-insensitive unique names, membership changes and their audit events (SCRUM-149).
tests/integration/routes/approvalSettings.test.js	30	Organization-wide and per-asset approval modes, including that an auto-approving asset cannot let an ineligible member skip the group check (SCRUM-148, SCRUM-150).

Iteration 3 changes and key test files

The layout and frameworks are unchanged, and so is the Vitest configuration of both packages apart from the AI-usage header added to the backend one. The backend suite grew from 45 to 48 test files (21 unit, 27 integration) and the frontend from 36 to 40. The new backend files are the three SCRUM-206 telemetry suites; the new frontend files are AiSearchPage, AiSearchAlert, aiSearchStatus and LoanTimeline. Foundry is still mocked in every automated case: the new client cases exercise the retry, the Retry-After wait and the shared deadline with the HTTP call mocked.
The new and most changed test files of Iteration 3:

Test file	Cases	What it proves
tests/integration/security/routePermissions.test.js	234	229 cases in Iteration 2. The new GET /api/search-telemetry route was picked up by the route-table walk with no change to the test, so it is checked against the permission matrix like every other route (SR-1).
tests/integration/routes/searchTelemetry.test.js	17	No row holds the query, a hash or encoding of it, or a user id; the summary is admin-only and returns aggregates only; a

		smuggled orgId is ignored (SR-2); the retention TTL index exists and a row cannot be backdated (SCRUM-206, SCRUM-242).
tests/unit/services/searchTelemetry.recording.test.js	17	One row per search and per alternatives lookup, with the right fallback reason; a rejected, throwing or hanging write never breaks or delays a search (SCRUM-206 AT-1, AT-6).
tests/unit/services/foundryClient.test.js	25	One retry on a 503, 429 or network error, never on a 400 or a timeout; never after the deadline; Retry-After read in seconds and capped at 3 s (SCRUM-244).
tests/unit/services/assetSearch.test.js	32	A sentence that matches nothing as a phrase still finds items by its words, ranked by how many words they hit; an outage falls back to word matches, not "No assets match" (SCRUM-243, SCRUM-244).
tests/unit/services/assetAlternatives.test.js	21	Same-category candidates first, and a fallback that never offers another category (SCRUM-184).
tests/unit/components/AiSearchPage.test.jsx (frontend)	10	Status line, chart, figures and fallback reasons render from the telemetry summary (SCRUM-242).
tests/unit/utils/aiSearchStatus.test.js (frontend)	9	The one rule behind both the page and the Overview alert: off, failing, catalog too large, or working normally.

4. Testing Metrics

4.1 Iteration 1

4.1.1 Test results and coverage

The test and coverage figures in the table below are from the CI run on main at commit e4a0efd (21 September 2026), which passed. The SPPP quality assurance plan sets line coverage at 80% or better; both packages are comfortably above it, and CI fails the build if either drops below.

Metric	Backend	Frontend

Test files	31 passed, 1 skipped (32)	27 passed, 1 skipped (28)
Test cases	679 passed, 6 skipped (685)	242 passed, 1 todo (243)
Test case pass rate	100% of executed cases	100% of executed cases
Line coverage (target >= 80%)	94.56%	95.97%
Statement coverage	94.46%	95.79%
Branch coverage	86.72%	89.97%
Function coverage	95.59%	95.26%
Dependency audit (SR-13)	0 vulnerabilities at high or above	0 vulnerabilities at high or above
Defects open against Iteration 1	SCRUM-125 (audit immutability, database layer) - open, blocked on Atlas console access	-

On the skipped and todo cases. The 6 skipped backend cases are tests/integration/security/auditDbRole.test.js, which verifies the insert-only database role for the audit collection (SR-8, SCRUM-125). They are skipped deliberately: they need a real authenticated MongoDB deployment, and run only when AUDIT_ROLE_TEST_URI points at one. They are not a gap in effort but an open item - until that role is applied, audit immutability rests on the application layer alone. The remaining skipped file and the single frontend todo are staged acceptance criteria for work not yet implemented, per the convention above.

4.1.2 Code structure metrics

These metrics were measured separately by the QA Lead from the source tree. LOC counts exclude comments and blank lines, and the per-file figures also exclude configuration files. Fan-out counts relative imports of the project's own modules and excludes third-party libraries.

Metric	Backend	Frontend	Combined
Average LOC / file	61.9 63 files, 3,902 LOC	63.4 55 files, 3,487 LOC	62.6 118 files, 7,389 LOC
Average LOC / function	13.6 260 functions	29.4 120 functions	18.5 380 functions
Average fan-out / file	2.6 165 internal import edges	3.1 168 internal import edges	2.8 333 internal import edges

4.2 Iteration 2

4.2.1 Test results and coverage

The backend figures in the table below are from the CI run on main at commit 5c4bcc8 (4 October 2026), which passed every job. The frontend figures are from a run of the same commit's suite with coverage, and match the totals recorded on PR #63. Both packages stay above the 80% line-coverage gate, and every coverage measure rose from Iteration 1.

Metric	Backend	Frontend
Test files	44 passed, 1 skipped (45)	35 passed, 1 skipped (36)

Test cases	1,114 passed, 6 skipped (1,120)	394 passed, 1 todo (395)
Test case pass rate	100% of executed cases	100% of executed cases
Line coverage (target >= 80%)	95.19%	97.79%
Statement coverage	95.13%	97.78%
Branch coverage	87.81%	93.08%
Function coverage	96.76%	97.76%
Dependency audit (SR-13)	0 vulnerabilities at high or above	0 vulnerabilities at high or above
Defects fixed in Iteration 2	PR #64: approval-queue rows showed "Unknown user" / "Unknown asset" after the UI rework; fixed, with 9 regression cases in requests.test.js	-
Defects open against Iteration 2	SCRUM-125 (audit immutability, database layer) - still open, blocked on Atlas console access	-

On the skipped and todo cases. These are unchanged from Iteration 1. The 6 skipped backend cases are still auditDbRole.test.js: the suite's in-memory MongoDB runs without authentication, so the insert-only audit role cannot be checked there, and the role has not yet been applied to any deployment (SCRUM-125). Until it is, audit immutability (SR-8) rests on the application layer, which is tested. The frontend skip and todo are staged acceptance criteria.

4.2.2 Code structure metrics

Measured from the source tree at commit 5c4bcc8 by script (a JavaScript parser), using the definitions in 4.1.2 over every .js and .jsx file under src: LOC excludes comments and blank lines, fan-out counts relative imports of the project's own modules, and LOC per function counts top-level functions, with callbacks nested inside a function counted toward it. The script counts slightly differently from the hand measurement in 4.1.2, so the comparison in 4.3 uses the same script run on the Iteration1 tag, comparing like with like.

Metric	Backend	Frontend	Combined
Average LOC / file	80.9 69 files, 5,579 LOC	74.0 71 files, 5,254 LOC	77.4 140 files, 10,833 LOC
Average LOC / function	12.2 336 functions	26.5 171 functions	17.0 507 functions
Average fan-out / file	2.9 197 internal import edges	3.2 228 internal import edges	3.0 425 internal import edges

4.3 Iteration 1 vs Iteration 2

The two iterations side by side. Each cell shows Iteration 1 then Iteration 2, with the change underneath. Test and coverage figures come from the two CI runs (e4a0efd and 5c4bcc8); the code-structure rows marked * use the script figures for both iterations, as explained in 4.2.2.

Metric	Backend	Frontend

Test files	32 → 45 +13	28 → 36 +8
Test cases	685 → 1,120 +435	243 → 395 +152
Test case pass rate	100% → 100% no change	100% → 100% no change
Line coverage	94.56% → 95.19% +0.63 pts	95.97% → 97.79% +1.82 pts
Statement coverage	94.46% → 95.13% +0.67 pts	95.79% → 97.78% +1.99 pts
Branch coverage	86.72% → 87.81% +1.09 pts	89.97% → 93.08% +3.11 pts
Function coverage	95.59% → 96.76% +1.17 pts	95.26% → 97.76% +2.50 pts
High or critical vulnerabilities	0 → 0 no change	0 → 0 no change
Source files	63 → 69 +6	55 → 71 +16
Average LOC / file *	62.9 → 80.9 +18.0	64.5 → 74.0 +9.5
Average LOC / function *	11.9 → 12.2 +0.3	26.1 → 26.5 +0.4
Average fan-out / file *	2.6 → 2.9 +0.3	2.9 → 3.2 +0.3

Iteration 2 added 435 backend and 152 frontend test cases while every coverage measure rose, so the new code arrived with proportionally more tests than the old. Average file size grew by about a fifth, mostly because Iteration 2 features were added to existing modules (checkout.service.js in particular); function length and fan-out barely moved.

4.4 Iteration 3

4.4.1 Test results and coverage

The figures below are from runs of each package's suite with coverage at commit ebfae65 (main, 11 October 2026), with the same configuration CI uses. The backend suite was run on the QA Lead's machine (Node 22.19), against the in-memory MongoDB replica set as in CI. The frontend suite was run separately; the same run of the Iteration 2 commit reproduced every Iteration 2 frontend figure exactly, so the two are directly comparable. The CI run on main at ebfae65 (GitHub Actions run 38169866232) also passed every job. Lint, the format check and npm audit were run on both packages at ebfae65 and passed.

Metric	Backend	Frontend
Test files	47 passed, 1 skipped (48)	39 passed, 1 skipped (40)
Test cases	1,193 passed, 6 skipped (1,199)	452 passed, 1 todo (453)
Test case pass rate	100% of executed cases	100% of executed cases

Line coverage (target >= 80%)	95.68%	96.34%
Statement coverage	95.62%	96.21%
Branch coverage	88.27%	88.99%
Function coverage	96.92%	96.67%
Dependency audit (SR-13)	0 vulnerabilities at high or above	0 vulnerabilities at high or above
Defects fixed in Iteration 3	SCRUM-184: similar items recommended unrelated assets (a camera for a laptop). SCRUM-244: AI searches failing on staging under Foundry rate limits (429). Both fixed with new cases, most confirmed failing before the fix.	SCRUM-240 (MT-21 Partial): request dates displayed a day early west of UTC; fixed, with conversion and display tests across both daylight-saving changes.
Defects open against Iteration 3	SCRUM-125 (audit immutability, database layer): auditDbRole.test.js is unchanged apart from its header, so the insert-only role is presumably still unapplied; confirm in Jira.	MT-19 Partial (dashboard Available figure lags after maintenance): no Iteration 3 change addressed it; re-tested as MT-49.

On the skipped and todo cases. The frontend skip and todo are the same staged acceptance criteria as before. auditDbRole.test.js is unchanged apart from its header, and its 6 backend cases are still the only skipped ones (SCRUM-125, SR-8).
On the frontend coverage dip. Iteration 3 added 58 frontend cases, but the UI redesign (SCRUM-241) added code faster than tests, so every frontend coverage measure fell for the first time: lines from 97.79% to 96.34% and branches from 93.08% to 88.99%. Both remain well above the 80% gate. The gap is concentrated in the redesign's new interactive components, which have no test file of their own: Drawer (69.7% of lines), RequestBoard (83.3%) and CommandPalette (85.1%). MT-44, MT-46 and MT-47 cover them by hand this iteration; dedicated component tests are the obvious next step.

4.4.2 Code structure metrics

Measured from the source tree at commit ebfae65 by script, with the definitions in 4.1.2 and 4.2.2. The Iteration 2 script was not kept in the repository, so a new one was written (a JavaScript parser). It reproduces the hand-measured Iteration 1 LOC exactly (3,902 backend, 3,487 frontend) and the Iteration 2 fan-out exactly (197 and 228 import edges), but counts slightly fewer LOC and functions than the Iteration 2 script (for example 5,510 rather than 5,579 backend LOC at 5c4bcc8). The comparison in 4.5 therefore uses the new script on both commits, comparing like with like. It counts the top-level functions of each file, including methods of top-level objects and classes, with nested callbacks counted toward their enclosing function.

Metric	Backend	Frontend	Combined
Average LOC / file	80.0 74 files, 5,921 LOC	80.0 83 files, 6,637 LOC	80.0 157 files, 12,558 LOC
Average LOC / function	12.8 340 functions	30.7 189 functions	19.2 529 functions
Average fan-out / file	2.9 211 internal import edges	3.2 262 internal import edges	3.0 473 internal import edges

4.5 Iteration 2 vs Iteration 3

Each cell shows Iteration 2 then Iteration 3, with the change underneath. Frontend test and coverage figures come from runs of 5c4bcc8 and ebfae65 made the same way; the backend figures come from the CI run at 5c4bcc8 and a run of the suite with coverage at ebfae65, under the same configuration. The code-structure rows marked * use the new script for both iterations, as explained in 4.4.2.

Metric	Backend	Frontend
Test files	45 → 48 +3	36 → 40 +4
Test cases	1,120 → 1,199 +79	395 → 453 +58
Test case pass rate	100% → 100% no change	100% → 100% no change
Line coverage	95.19% → 95.68% +0.49 pts	97.79% → 96.34% −1.45 pts
Statement coverage	95.13% → 95.62% +0.49 pts	97.78% → 96.21% −1.57 pts
Branch coverage	87.81% → 88.27% +0.46 pts	93.08% → 88.99% −4.09 pts
Function coverage	96.76% → 96.92% +0.16 pts	97.76% → 96.67% −1.09 pts
High or critical vulnerabilities	0 → 0 no change	0 → 0 no change
Source files	69 → 74 +5	71 → 83 +12
Average LOC / file *	79.9 → 80.0 +0.1	72.9 → 80.0 +7.1
Average LOC / function *	12.8 → 12.8 no change	28.7 → 30.7 +2.0
Average fan-out / file *	2.9 → 2.9 no change	3.2 → 3.2 no change

Iteration 3 added 79 backend and 58 frontend test cases. On the backend every coverage measure rose slightly, as in Iteration 2, and the new telemetry model, repository, service and route left file size, function length and fan-out where they were. On the frontend, the redesign grew the source faster than the tests: files rose by 12, average file size by about a tenth and average function length by two lines, and coverage fell for the first time while staying well above the 80% gate (4.4.1).

5 Requirements Traceability (TODO: run remaining MTs)

5.1 Functional requirements

Note. The SPPP does not number its functional requirements, they're numbered here for convenience

ID	Requirement	System tests	Result	Status
FR-1	Org Admin creates an organization portal (Essential)	MT-02	Pass	Covered
FR-2	Members browse, check out and return items (Essential)	MT-06, MT-10, MT-11, MT-12, MT-13, MT-28, MT-29, MT-31, MT-40	Pass; MT-40 Partial (Catalog shows only the first 25 assets, defect to be logged)	Covered, open defect
FR-3	Audit log of requests, returns and inventory changes (Essential)	MT-16, MT-17	Pass	Covered
FR-4	Login, and a dashboard of one's own requests (Essential)	MT-01, MT-03, MT-14, MT-35	Pass	Covered
FR-5	Admin dashboard and inventory management (Essential)	MT-07, MT-08, MT-09, MT-15, MT-18, MT-19, MT-44, MT-48	Pass; MT-19/MT-48 Partial (dashboard Available count lags after maintenance, defect open)	Covered, open defect
FR-6	Admin-configurable approval flows (Desirable)	MT-11, MT-24, MT-25, MT-43	Pass	Covered
FR-7	User groups (Desirable)	MT-26, MT-46	Pass	Covered
FR-8	Equipment restricted to certain groups (Desirable)	MT-27	Pass	Covered
FR-9	AI-assisted search and recommendations (Desirable)	MT-32, MT-33, MT-38, MT-39, MT-41, MT-42	Pass	Covered
FR-O1 to FR-O4	Optional: AI reliability scoring, AI depreciation forecast, TypeScript conversion, purchase-order requests	None	n/a	Excluded: not built in Iterations 1–3

5.2 Security requirements

ID	Requirement	System tests	Result	Status
SR-1	Role-based access enforced server-side	MT-04, MT-18, MT-20, MT-30, MT-45; routePermissions.test.js (234 cases)	Pass	Covered
SR-2	Data isolated between organizations, including via the API	MT-05, MT-34, MT-47; tenantIsolation.test.js (27 cases)	Pass	Covered
SR-3	Passwords hashed with bcrypt/argon2, never stored or logged in plain text	auth.test.js (no passwordHash returned), users.test.js, changePassword.test.js	Pass (CI, ebfae65)	Covered
SR-4	Short-lived JWTs, refresh tokens, httpOnly cookies, idle expiry	MT-01; auth.test.js (HttpOnly cookies, idle-expired token rejected)	Pass	Covered
SR-5	HTTPS only, no HTTP fallback	env.test.js (refuses non-https base URL in production); weekly OWASP ZAP scan of staging; MT-49	Pass	Covered
SR-6	Input validated against NoSQL injection and stored XSS	Operator-injection cases in auth, audit and users tests; validate.test.js; MT-50	Pass	Covered
SR-7	Sensitive PII encrypted at rest	None	n/a	Excluded: only names and emails are stored, so the condition does not apply
SR-8	Audit log cannot be edited or deleted through UI or API	auditEvent.test.js ("exposes only append() and query()"); routePermissions.test.js. Database-layer check auditDbRole.test.js is skipped (SCRUM-125); MT-51	Pass	Covered
SR-9	Every state change logged with actor, time and before/after	MT-16, MT-17; audit assertions in assets, custodyConfirmation and approvalSettings tests	Pass	Covered
SR-10	Audit log detailed enough to settle disputes	MT-16 (chain of custody), MT-28 (history shows who recorded the pickup)	Pass	Covered

SR-11	Secrets in environment variables, never committed	env.test.js (startup refuses missing or insecure secrets); MT-52	Pass	Covered
SR-12	Login endpoints rate-limited	auth.test.js ("returns 429 once the auth rate limit is exceeded")	Pass	Covered
SR-13	Dependencies scanned in CI	CI "Dependency audit (SR-13)" step, both packages	0 high/critical (4.4.1)	Covered
SR-14	CORS limited to allowlisted origins	app.test.js ("CORS reflects only allowlisted origins")	Pass	Covered

5.3 Non-functional requirements

ID	Requirement	System tests	Result	Status
NFR-1	99% uptime during business hours	None	n/a	Excluded: needs production monitoring over time; staging runs on free hosting tiers. Team to confirm.
NFR-2	Failures never leave inventory in a partial state	Rollback cases in groups, users and organizations tests; checkout.transitions.test.js	Pass	Covered
NFR-3	Scheduled backups with a documented recovery process	backup.yml workflow; restore procedure in code/docs/backups.md; MT-53	Pass	Covered
NFR-4	Many organizations at once without slowdown; data indexed by org	tenantIsolation.test.js; org-scoped indexes from migrations	Pass (isolation only)	Excluded (load part): the application is running on a free render backend so the load is limited. Not worth measuring.
NFR-5	Responsive with [TBD] concurrent users per org	None	n/a	Excluded: Again, the service is running on a free render backend. The resources

				are intentionally limited as part of this free tier, so this is not worth measuring. It would measure the limitations of the free tier more than anything.
NFR-6	API responses under 500 ms	MT-54	n/a	Covered
NFR-7	Pages load in 2–3 s	MT-54	n/a	Covered
NFR-8	Works on the latest two versions of Chrome, Firefox, Safari and Edge	MT-55	Pass	Covered
NFR-9	Usable on desktop, tablet and phone widths	MT-56	Pass	Covered
NFR-10	Style guide enforced in CI	CI "Lint (NFR-10)" and "Format check (NFR-10)" steps	Pass (ebfae65)	Covered
NFR-11	Automated tests for approval and checkout logic	checkout.transitions.test.js, approvalPolicy.test.js, approvalSettings.test.js	Pass	Covered
NFR-12	WCAG 2.1 AA where practical	MT-45, MT-46 (keyboard use and focus), MT-57	Pass, mostly (only minor issue from lighthouse test regarding heading hierarchy)	Covered
NFR-13	New users finish core workflows without documentation	MT-58	Pass	Covered
NFR-14	Data minimization (FERPA review if used by a school)	MT-37 (telemetry stores no query text or user id)	Pass	Covered (data minimization). FERPA review excluded: not deployed for a real institution.

5.4 Manual test cases added for traceability

These close the gaps found in 2.7.1–2.7.3. They use the environment and seed in 2.2 and 2.3, or the staging deployment where stated. Results are left for the QA Lead to record when the cases are run.

5.4.1 Test case summary

ID	Test case	Type	Priority	Result
MT-49	HTTPS only: no page or data served over HTTP (SR-5)	System (security)	High	Pass
MT-50	Stored XSS in asset fields is shown as text (SR-6)	System (security)	High	Pass
MT-51	Audit entries cannot be edited or deleted (SR-8)	System (security)	High	Pass
MT-52	No secrets in the repository history (SR-11)	System (security)	Medium	Pass (only false positives detected)
MT-53	Restore a backup into a separate database (NFR-3)	System	Medium	Pass
MT-54	API response and page load times on staging (NFR-6, NFR-7)	System	Medium	Pass
MT-55	Core workflows in Chrome, Firefox, Safari and Edge (NFR-8)	System	Medium	Pass
MT-56	Core workflows at phone and tablet widths (NFR-9)	System	Medium	Pass
MT-57	Accessibility audit with Lighthouse (NFR-12)	System	Low	Mostly Pass (only the catalog page scored less than 100. It got a 98, the one penalty being “Heading elements are not in a sequentially-descendin g order”
MT-58	First-time user completes a request and approval (NFR-13)	Acceptance	Low	Pass (after figuring out Catalog and Requests were the right pages to use, the rest was fairly intuitive)

5.4.2 Test steps

ID	Steps	Expected result
MT-49	1. Open the staging frontend and backend /health using http:// instead of https://. 2. Inspect the response headers of any API call.	Step 1: both redirect to https:// or refuse; no page or data is served over http. Step 2: a Strict-Transport-Security header is present.

MT-50	1. As admin@a.test, create an asset named <img src=x onerror=alert(1)> with the same text as its description. 2. As member@a.test, open the Catalog and the asset's page. 3. As admin@a.test, open the audit log entry for the creation.	Steps 2–3: the text is shown literally; no alert appears and no image renders.
MT-51	1. As admin@a.test, open Audit log and an asset's history; look for any edit or delete control. 2. In the browser console, send PATCH and DELETE requests to an audit entry's URL.	Step 1: no edit or delete control exists. Step 2: both requests are refused (404 or 405); the entry is unchanged on reload.
MT-52	1. Run a secret scan over the repository's full history (GitHub secret scanning or gitleaks). 2. Confirm .env files are listed in .gitignore.	No API keys, database credentials or JWT secrets in any commit.
MT-53	1. Take the latest backup produced by the backup workflow. 2. Restore it into a separate database following code/docs/backups.md. 3. Point a local backend at the restored database and sign in as admin@a.test.	Step 2: the restore completes without errors. Step 3: assets, requests and audit history match the source.
MT-54	1. With the browser network panel open, sign in on staging as member@a.test and load the Catalog, My requests and an asset page. 2. Note each API call's time and each page's load time (warm backend; repeat 3 times).	API responses under 500 ms; pages fully loaded within 3 s.
MT-55	Run MT-01, MT-10 and MT-12 in current Chrome, Firefox, Safari and Edge.	Same results in every browser; no console errors or broken layout.
MT-56	1. Using browser device emulation at 390 px (phone), sign in as member@a.test, open the Catalog and request an asset. 2. As approver@a.test, approve it. 3. Repeat at 820 px (tablet).	Everything is readable and usable with no horizontal scrolling or hidden controls.
MT-57	Run Lighthouse (Accessibility) on the login page, Catalog, an asset page and Approvals.	No contrast or missing alt-text failures; score recorded per page.
MT-58	1. Give someone outside the team member@a.test's login and ask them, with no instructions, to request a camera. 2. Ask them, signed in as approver@a.test, to approve it.	They finish both tasks without help; note anything they got stuck on.

6. AI Usage Log

This table summarizes AI contributions in this document.

Section	Who	AI contribution (%)	Description	Verified by
1, 3, 4	Amber (on behalf of QA)	75%	Claude Code compiled the automated testing report and the metrics table from the CI run on main and from the test suite itself. Every figure was read from the CI log rather than estimated. The manual testing report was deliberately left unfilled.	Alex Picard
4.2 and layout	Orelmis	20%	The QA Lead measured the LOC, function-length and fan-out figures. Claude formatted them into a table, drafted the surrounding text, and reworked the document layout; it did not generate or estimate any figures.	Orelmis Toribio
Iteration 2 additions (1-4)	Orelmis Toribio	85%	Claude compiled the Iteration 2 sections from the repository history since the Iteration1 tag and the CI run on main at 5c4bcc8. It read the backend figures from the CI log, ran the frontend suite with coverage, measured the code-structure metrics with a script, and drafted the Iteration 2	Orelmis Toribio

			manual test cases. It did not run the manual tests; their results are recorded by the QA Lead.	
Iteration 3 additions (1-4, 6, 7)	Orelmis Toribio	85%	Claude compiled the Iteration 3 sections from the repository history since the Iteration2 tag. It read the backend run made on the QA Lead's machine at ebfae65; ran the frontend suite with coverage at ebfae65, and at 5c4bcc8 to confirm the Iteration 2 figures reproduce; ran lint, the format check and npm audit on both packages; measured the code-structure metrics with a new script checked against the Iteration 1 and 2 figures; and drafted manual test cases MT-36 to MT-49. It did not run the manual tests.	Orelmis Toribio

7. References

[1] Team 3, "SafeDrop Software Project Proposal and Plan (SPPP)," CS673, Boston University, Sept. 2026.
[2] Team 3, "SafeDrop Software Design Document (SDD) v0.2," CS673, Boston University, Sept. 2026.
[3] SafeDrop CI run on main at commit e4a0efd, GitHub Actions, 21 Sept. 2026 - the source of every figure in Sections 3 and 4 except the code structure metrics.
[4] code/docs/staged-acceptance-tests.md - the convention for staging acceptance criteria as tests.
[5] code/docs/audit-db-role.md - the insert-only database role for the audit collection (SCRUM-125).
[6] SafeDrop CI run on main at commit 5c4bcc8, GitHub Actions, 4 Oct. 2026 - the source of the Iteration 2 backend figures in Sections 1, 3 and 4.
[7] code/docs/std-scrum-151-test-cases.md - the SCRUM-151 test-case summary (29 automated cases).
[8] SafeDrop CI run on main at commit ebfae65, GitHub Actions run 38169866232, 11 Oct. 2026 - every job passed. The Iteration 3 backend figures come from a run of the same commit's suite with coverage

(4.4.1).
[9] code/docs/od-9-search-telemetry.md - OD-9, what SearchQueryLog may store (SCRUM-206).
[10] code/docs/od-10-ai-candidate-retrieval.md - OD-10, the candidate-retrieval spike (SCRUM-191).

8. Glossary

Term	Definition
Auto-approve	An approval setting under which a request is approved as soon as it is submitted; the item is still handed over in person.
Candidate cap	The most catalogue items one AI search sends the model: 100. A search that hits it is counted, because some items were left out of it.
Command palette	The ⌘K / Ctrl+K "jump to" box: type part of a page's or an asset's name and press Enter. It offers only pages the viewer's role can open.
Coverage gate	A CI rule that fails the build when line coverage falls below the agreed target (80% here).
Custody confirmation	The rule that a change of custody is recorded by the side that can verify it: the borrower may record a pickup, but a return is closed by someone else.
Fallback (search)	Plain text matching used when AI search is off, unavailable, or returns an answer the app cannot use; members still get results, labelled "Showing text matches".
Fan-out	The number of the project's own modules that a source file imports; a measure of how tightly a file is coupled to the rest of the codebase. Third-party libraries are not counted.
Integration test	A test that exercises several layers together, here the API against a real database.
LOC (lines of code)	Non-blank, non-comment source lines.
Microsoft Foundry	The hosted AI model service behind AI-assisted search and recommendations, called only from the backend.
mongodb-memory-server	A library that starts a real MongoDB replica set in memory for the duration of a test run.
Pickup window	How long after the start of a request an approved item may be collected before the approval expires.
Regression test	A test kept in the suite so that a behaviour which once worked cannot silently break.
Request board	The requests view with a column per stage

	(waiting for a decision, ready for pickup, out on loan, done) and one-click next steps on each card.
Restricted equipment	An asset only members of named user groups may request.
Search telemetry	One SearchQueryLog row per AI search or similar-items lookup, recording how it behaved but never the query text or who searched; kept 90 days.
Staged acceptance test	A bodyless test case standing in for an acceptance criterion not yet implemented; Vitest reports it as todo.
Supertest	A library for calling Express routes in tests without starting an HTTP server.
Unit test	A test of one module with its dependencies mocked.
User group	A named set of members within an organization, used to restrict equipment.
Vitest	The test runner used for both packages.